Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшегообразования
«Сибирский государственный университет телекоммуникаций и информатики»
Уральский технический институт связи и информатики (филиал) в г. Екатеринбурге
ЦК ИТ и АСУ кафедры ИСТ

Отчет
по учебной практике
по МДК 02.02 «Инструментальные средства разработки программного обеспечения»
на тему «Разработка программного модуля для учета заявок на ремонт бытовой техники»

студента 3 курса 381 группы

Фамилия Жулдыбин

Имя, отчество Вячеслав Александрович

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы и программирование»

г. Екатеринбург 2026г.

Отзыв руководителя

Содержание

Введение 4
1 Анализ предметной области 6
1.1 Предметная область 6
1.2 Целевая аудитория 6
1.3 Требования к программному обеспечению 7
2 Этап проектирования базы данных 9
2.1 Описание базы данных 9
2.2 ER-диаграмма 10
3 Этап разработки базы данных 11
3.1 Описание базы данных 11
3.2 Схема разработанной базы данных 12
3.3 Заполнение базы данных тестовыми данными 12
3.4 Администрирование и защита БД 13
4 Разработка дизайна 15
5 Описание дипломного проекта 19
5.1 Обоснование выбора инструментальных средств разработки 19
5.2 Дизайн приложения 20
Заключение 23
Приложение А 24

Введение

На фоне повсеместной цифровизации и активного развития сферы услуг, современные сервисные центры всё чаще сталкиваются с потребностью в автоматизации учета и обработки заявок на ремонт бытовой техники. Внедрение профильных информационных систем становится крайне актуальным, так как оно помогает не только рационализировать труд сотрудников, но и заметно улучшить клиентский сервис. Ключевым условием успешной работы сервисных компаний сегодня выступает автоматизация таких процессов, как прием обращений, распределение работы между мастерами, мониторинг этапов ремонта и генерация аналитики.

Учитывая сказанное, создание программного модуля для управления заявками на ремонт является востребованным решением, цель которого — рост производительности персонала и улучшение коммуникации с заказчиками. Разрабатываемый продукт сочетает в себе понятный графический интерфейс, систему разделения прав для разных групп пользователей (администратор, менеджер, клиент) и безопасное хранение информации в реляционной БД. Это обеспечивает эффективный контроль над каждым этапом работы с заявкой.

Функционал разрабатываемого модуля включает: регистрацию и вход в систему с учетом ролей, формирование и корректировку заявок, прикрепление исполнителей к заказам, мониторинг статусов ремонта, управление справочниками (клиенты, виды техники, мастера), а также создание отчетов по среднему времени выполнения работ. Важный акцент сделан на ролевой модели доступа: администратору доступны все функции, менеджер отвечает за назначение мастеров и обновление статусов, а клиент может самостоятельно оформлять заявки и следить за их ходом.

Целью учебной практики является разработка программного модуля для учета заявок на ремонт бытовой техники с расширенным функционалом для различных категорий пользователей. Для достижения поставленной цели необходимо выполнить следующие задачи:

– проанализировать требования к системе учета заявок и определить необходимый функционал;

– спроектировать структуру базы данных, обеспечивающую целостность и непротиворечивость хранимой информации;

– разработать графический интерфейс пользователя с учётом разделения прав доступа;

– реализовать функционал для администратора (полное управление системой);

– реализовать функционал для менеджера (назначение мастеров и управление статусами);

– реализовать функционал для клиента (создание и отслеживание собственных заявок);

– выполнить интеграцию с СУБД PostgreSQL для хранения данных;

– провести тестирование разработанного модуля и оформить документацию.

Разработка и внедрение программного модуля для учета заявок на ремонт бытовой техники позволит сервисному центру автоматизировать ключевые бизнес-процессы, минимизировать вероятность ошибок при ручном вводе данных, обеспечить прозрачность выполнения заказов для клиентов и повысить общую эффективность работы предприятия.

1 Анализ предметной области

1.1 Предметная область

Функционирование предприятий по ремонту бытовой техники требует выстроенной системы управления. Поскольку бытовые приборы играют ключевую роль в обеспечении комфорта современного человека, спрос на их своевременное обслуживание остается стабильно высоким. При этом традиционный (ручной) учет обращений посредством бумажных журналов и неструктурированных заметок демонстрирует низкую эффективность: он влечет за собой риск потери информации, ошибки персонала и снижение общего качества клиентского сервиса. В сложившейся ситуации разработка специализированного программного модуля выступает необходимой и актуальной мерой. Данное решение позволит уйти от рутинных операций, полностью автоматизировав процессы приема, распределения и контроля ремонтных заявок.

1.2 Целевая аудитория

Целевая аудитория программного модуля для учета заявок на ремонт бытовой техники включает в себя несколько ключевых групп пользователей, каждая из которых имеет свои уникальные потребности и требования. Основные группы целевой аудитории представлены ниже:

Клиенты:

– самостоятельная регистрация заявок - это возможность легко обращаться в сервисный центр, предоставляя первичную информацию о неисправности.

– отслеживание статуса ремонта - это доступ к актуальной информации о ходе выполнения заказа и контроль процесса.

– согласование условий - это возможность взаимодействовать с сервисным центром по вопросам получения результатов и стоимости работ.

Менеджеры:

– менеджеры отвечают за прием и обработку заявок, играя важную связующую роль, и имеют свои потребности;

– распределение задач нужно для возможности оптимально назначать мастеров на заявки с учетом их узкой специализации (например, работа с гидравлическими системами или электроникой);

– контроль сроков и выполнения – это инструменты для отслеживания изменения статусов по каждой заявке;

– коммуникация с клиентами - это возможность консультировать заказчиков, информировать их о ходе ремонта и согласовывать детали.

Мастера:

– доступ к рабочим задачам нужно для возможности видеть актуальный список назначенных заявок, требующих проведения диагностики и ремонта;

– фиксация результатов.

Администраторы сервисного центра:

– управление пользователями, создание, редактирование учетных записей менеджеров, мастеров и клиентов, а также управление их правами доступа;

– ведение справочной информации, добавление и актуализация данных о типах ремонтируемой техники, списке мастеров и вариантах статусов;

– отчетность и аналитика, сбор статистики работы сервисного центра на основе накопленных данных для внутреннего анализа и принятия управленческих решений.

1.3 Требования к программному обеспечению

Функциональность: система должна обеспечивать регистрацию и вход пользователей на основе ролевой модели. Основной функционал включает полный цикл управления заявками (добавление, изменение, удаление, смена статуса, назначение исполнителя), работу со справочниками, поиск данных и формирование сводной статистики.

Удобство использования: требуется разработать интуитивно понятный и стандартизированный интерфейс с логичной навигацией. Система должна информировать пользователя об ошибках и запрашивать подтверждение при выполнении необратимых операций.

Надежность хранения: использование реляционной БД должно обеспечивать целостность и непротиворечивость данных (включая ссылочную целостность). Обязательно наличие функционала для резервного копирования и восстановления.

Безопасность: приложение должно защищать персональные данные, использовать хеш-функции для паролей и блокировать несанкционированный доступ к информации за счет строгого ролевого разграничения.

Масштабируемость: архитектура модуля должна быть спроектирована с учетом возможного внедрения новых функций и увеличения нагрузки (рост базы заявок и числа пользователей) без потери скорости работы.

Производительность: требуется оптимизация запросов к БД для мгновенного отклика интерфейса при выполнении стандартных пользовательских сценариев даже в условиях больших объемов данных.

Обработка ошибок: при возникновении исключительных ситуаций система должна корректно прерывать операции, выполнять откат транзакций и выдавать пользователю понятные рекомендации по устранению проблемы.

Документирование: код программы должен быть прокомментирован и написан с соблюдением единых стандартов именования. В комплект поставки входят руководство пользователя и документация разработчика.

Исследование предметной области показало, что традиционный учет заявок в сервисном центре по ремонту техники неэффективен: ему не хватает прозрачности, инструментов контроля и аналитики. Управление таким процессом требует внедрения автоматизированных решений.

Разработка специализированного ПО закроет эту потребность: она позволит автоматизировать рутину, надежно защитить данные и выстроить работу разных групп пользователей через разграничение прав. Создаваемый программный модуль

должен строго соответствовать всем выдвинутым требованиям к функционалу, безопасности, эргономике и производительности.

2 Этап проектирования базы данных

2.1 Описание базы данных

Спроектированная база данных предназначена для централизованного хранения и обработки информации о заявках на ремонт, зарегистрированных клиентах, персонале сервисного центра и справочниках бытовой техники. Она выступает связующим звеном, обеспечивающим эффективное и оперативное взаимодействие между различными участниками системы.

Гарантия целостности и непротиворечивости данных реализуется на уровне СУБД за счет применения системы первичных и внешних ключей. Это исключает возможность появления дублирующихся или записей (например, заявок без привязки к клиенту). Безопасность системы подкрепляется механизмами защиты персональных данных сотрудников и клиентов, а также строгим разграничением прав, что предотвращает утечки и несанкционированное изменение информации о заказах.

Подводя итог, можно утверждать, что разработанная база данных является мощным ядром программного модуля. Она эффективно управляет потоками данных о заявках, распределении мастеров, этапах ремонта и статистике, обеспечивая высокую степень автоматизации учета и комфортную работу для всех пользователей сервисного предприятия.

2.2 ER-диаграмма

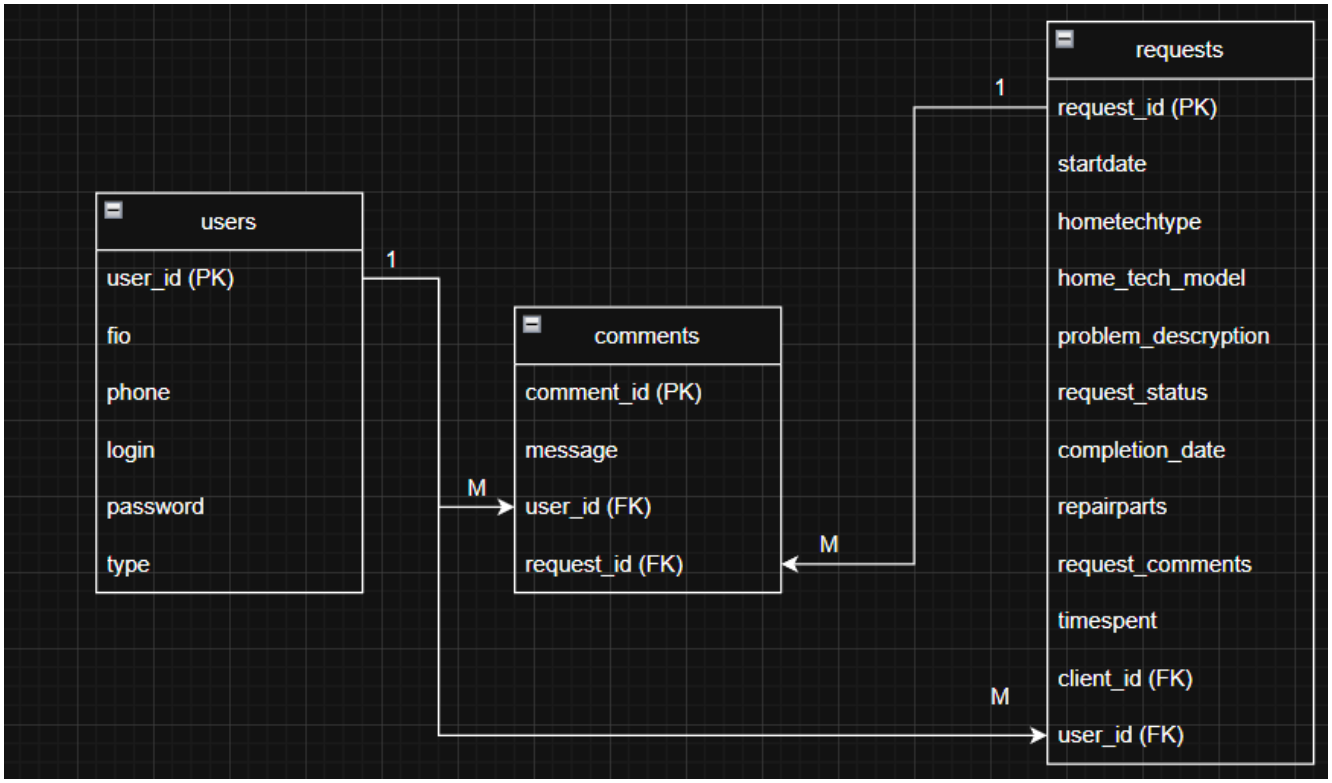

Рисунок 2.1 – ER-диаграмма

3 Этап разработки базы данных

3.1 Описание базы данных

Таблицы базы данных

Таблица «Пользователи» (users)

Содержит сведения обо всех пользователях системы.

– userid (INTEGER, PRIMARY KEY) – уникальный идентификатор пользователя;

– fio (VARCHAR (255)) – ФИО пользователя;

– phone (VARCHAR (20)) – номер телефона;

– login (VARCHAR (50)) – логин для входа в систему;

– password (VARCHAR (50)) – пароль;

– type (VARCHAR (50)) – тип (роль) пользователя в системе (например: клиент, менеджер, мастер).

Таблица «Заявки» (requests)

Содержит информацию о заявках на ремонт бытовой техники.

– requestid (INTEGER, PRIMARY KEY) – уникальный идентификатор заявки;

– startdate (DATE) – дата регистрации заявки;

– hometechtype (VARCHAR(100)) – вид бытовой техники;

– hometechmodel (VARCHAR(100)) – модель техники;

– problemdescription (TEXT) – описание неисправности;

– requeststatus (VARCHAR(50)) – статус заявки;

– completiondate (DATE) — планируемая или фактическая дата завершения ремонта;

– repairparts (TEXT) — информация об использованных или необходимых запчастях;

– masterid (INTEGER, FOREIGN KEY REFERENCES users(userid)) – назначенный мастер;

– clientid (INTEGER, FOREIGN KEY REFERENCES users(userid)) – клиент, создавший заявку;

– priority (VARCHAR(50)) – приоритет заявки;

– requestcomments (TEXT) – общие комментарии/заметки к заявке;

– timespent (VARCHAR(100)) – время, затраченное на выполнение ремонта.

Таблица «Комментарии» (comments)

– Содержит детализированные комментарии к заявкам (например, ход работы или переписка).

– commentid (INTEGER, PRIMARY KEY) – уникальный идентификатор комментария;

– message (TEXT) – текст комментария/сообщения;

– masterid (INTEGER, FOREIGN KEY REFERENCES users(userid)) – пользователь (мастер), оставивший комментарий;

– requestid (INTEGER, FOREIGN KEY REFERENCES requests(requestid)) – заявка, к которой относится данный комментарий.

3.2 Схема разработанной базы данных

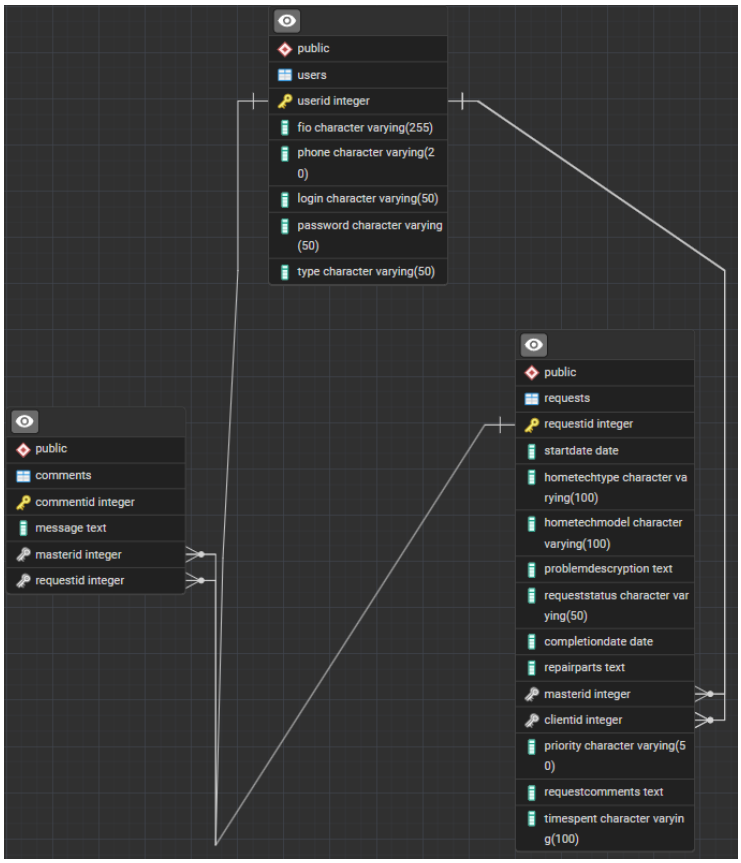

Рисунок 3.1 – Разработанная схема БД

3.3 Заполнение базы данных тестовыми данными

	commentid [PK] integer	message text	masterid integer	requestid integer
1	1	Интересная поломка	2	1
2	2	Очень странно, будем разбираться!	3	2
3	3	Скорее всего потребуется мотор обдува!	2	7
4	4	Интересная проблема	2	1
5	5	Очень странно, будем разбираться!	3	6

Рисунок 3.2 – Таблица коментариев

	requestid [PK] integer	startdate date	hometechtype character varying (100)	hometechmodel character varying (100)	problemdescryption text	requeststatus character varying (50)
1	0	2026-03-11	Пылесос	Урал	Кнопка включения	В процессе ремонта
2	1	2023-06-06	Фен	Ладомир TA112 белый	Перестал работать	Готова к выдаче
3	2	2023-05-05	Тостер	Redmond RT-437 черный	Перестал работать	Завершена
4	3	2022-07-07	Холодильник	Indesit DS 316 W белый	Не морозит одна из камер холодильника	Ожидание запчастей
5	4	2023-08-02	Стиральная машина	DEXP WM-F610NTMA/WW белый	[null]	Новая заявка
6	5	2023-08-02	Мультиварка	Redmond RMC-M95 черный	Перестала включаться	Готова к выдаче
7	6	2023-08-02	Фен	Ладомир TA113 чёрный	Перестал работать	Готова к выдаче
8	7	2023-07-09	Холодильник	Indesit DS 314 W серый	Гудит, но не замораживает	Готова к выдаче

Рисунок 3.3 – Таблица заявок (часть 1)

repairparts text	masterid integer	clientid integer	priority character varying (50)	requestcomments text	timespent character varying (100)
В ожидании запчастей	[null]	3	Средний	[null]	[null]
Pfvty`y ghjwtccjh c	2	7	Средний	[null]	[null]
[null]	3	7	Средний	[null]	[null]
[null]	2	8	Средний	[null]	[null]
[null]	2	8	Средний	[null]	[null]
[null]	[null]	9	Средний	[null]	[null]
	2	7	Средний	[null]	[null]
Мотор обдува морозильной камеры холодильника	2	8	Средний	[null]	[null]

Рисунок 3.4 – Таблица заявок (часть 2)

	userid [PK] integer	fio character varying (255)	phone character varying (20)	login character varying (50)	password character varying (50)	type character varying (50)
1	1	Трубин Никита Юрьевич	89210563128	kasoo	root	Менеджер
2	2	Мурашов Андрей Юрьевич	89535078985	murashov123	qwerty	Мастер
3	3	Степанов Андрей Викторович	89210673849	test1	test1	Мастер
4	4	Перина Анастасия Денисовна	89990563748	perinaAD	250519	Оператор
5	5	Мажитова Ксения Сергеевна	89994563847	krutiha1234567	1234567890	Оператор
6	6	Семенова Ясмина Марковна	89994563847	login1	pass1	Мастер
7	7	Баранова Эмилия Марковна	89994563841	login2	pass2	Заказчик
8	8	Егорова Алиса Платоновна	89994563842	login3	pass3	Заказчик
9	9	Титов Максим Иванович	89994563843	login4	pass4	Заказчик

Рисунок 3.5 - Таблица пользователей

3.4 Администрирование и защита БД

Для администрирования и защиты базы данных программного модуля учета заявок на ремонт бытовой техники необходимо учитывать несколько ключевых аспектов: управление доступом, резервное копирование, мониторинг и обеспечение безопасности данных. Рассмотрим каждый из этих аспектов более подробно.

Управление доступом

В разработанной системе реализована ролевая модель доступа, которая обеспечивает разграничение прав пользователей в соответствии с их функциональными обязанностями.

Роли и права доступа

В базе данных созданы следующие роли, соответствующие типам пользователей системы:

– администратор - обладает полными правами на управление всеми таблицами и данными;

– менеджер - имеет права на чтение и изменение заявок, назначение мастеров, но не имеет прав на удаление данных и управление клиентами;

– клиент - имеет ограниченные права на создание и просмотр только собственных заявок.

```
-- Создание ролей
CREATE ROLE administrator;
CREATE ROLE manager;
CREATE ROLE client_role;

-- Назначение прав для администратора
GRANT ALL PRIVILEGES ON DATABASE repair_service TO
administrator;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO
administrator;

-- Назначение прав для менеджера
GRANT SELECT, INSERT, UPDATE ON repair_requests, technicians,
request_statuses TO manager;
GRANT SELECT ON clients, device_types, users TO manager;

-- Назначение прав для клиента
GRANT SELECT ON repair_requests TO client_role;
GRANT INSERT ON repair_requests TO client_role;
```

В самом приложении права доступа реализованы на уровне кода: при входе в систему пользователь идентифицируется, и в зависимости от его роли формируется набор доступных функций и отображаемых данных.

4 Разработка дизайна

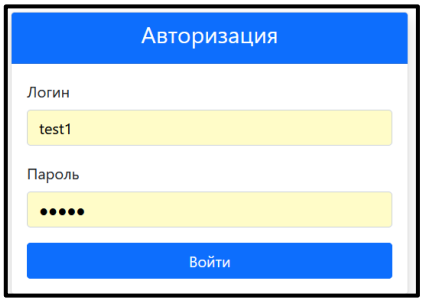

Рисунок 4.1 – Страница входа

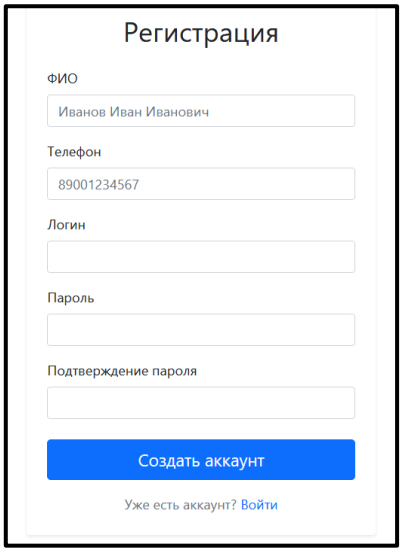

Рисунок 4.2 – Страница регистрации

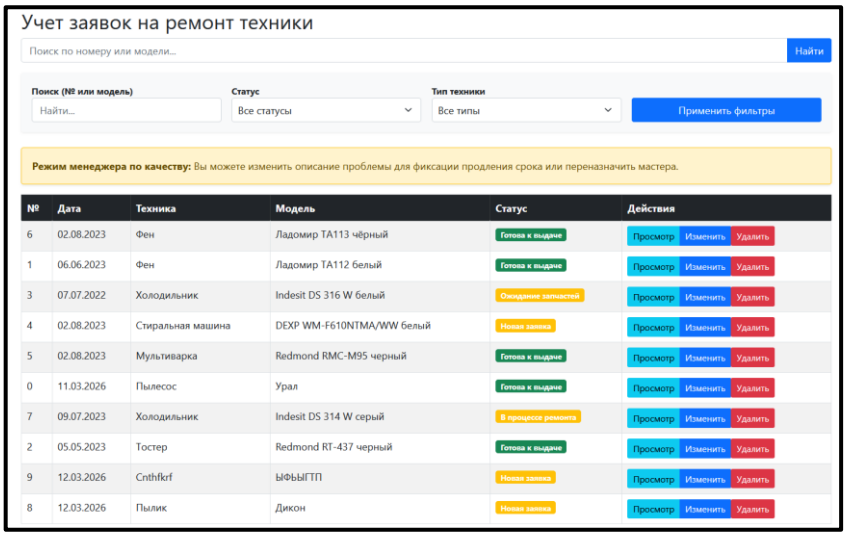

Рисунок 4.3 – Страница всех заявок менеджера

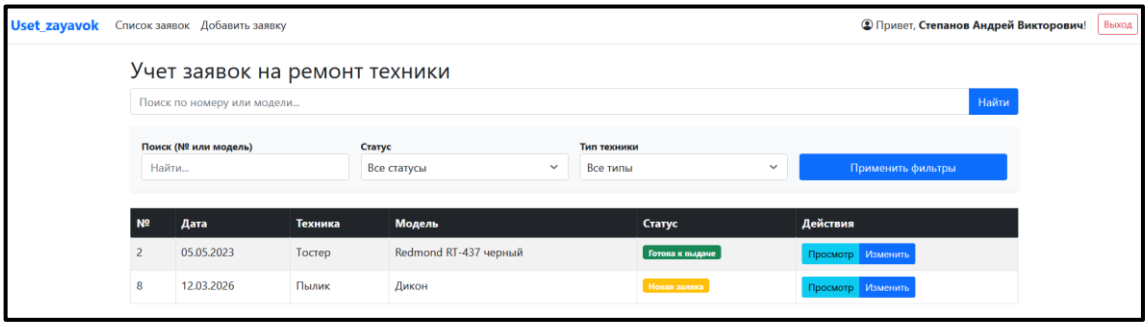

Рисунок 4.4 – Страница заявок мастера

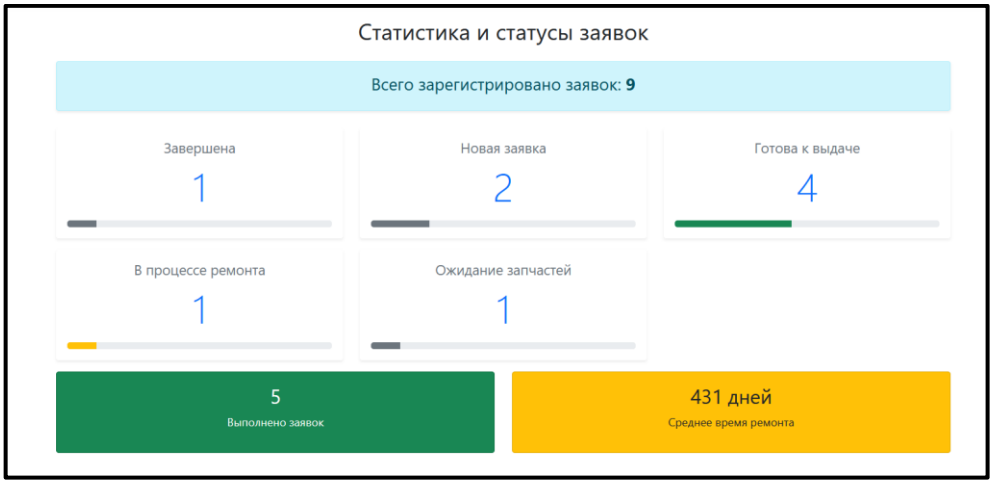

Рисунок 4.5 – Страница статусов заявок

Рисунок 4.6 – Эффективность сотрудников, которую видит менеджер

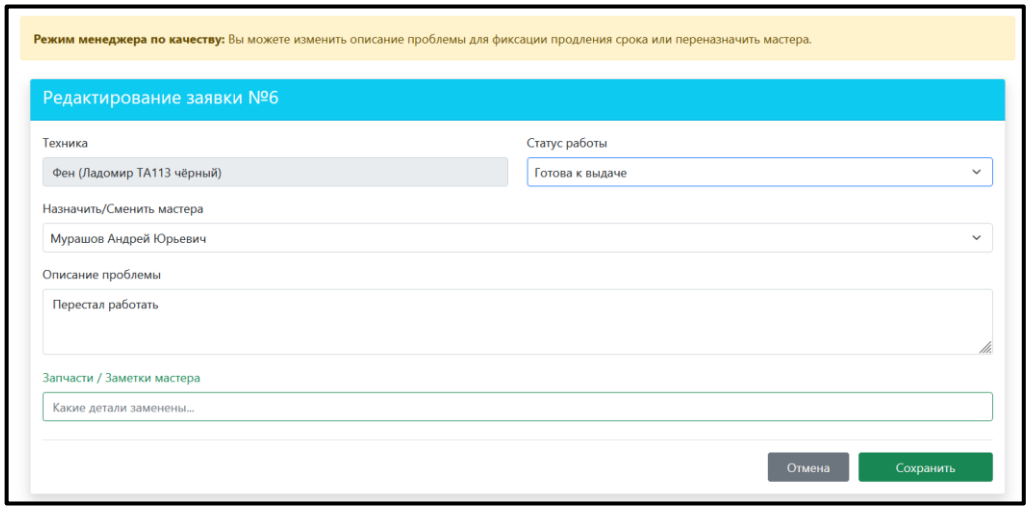

Рисунок 4.7 – Страница редактирования заявки менеджером

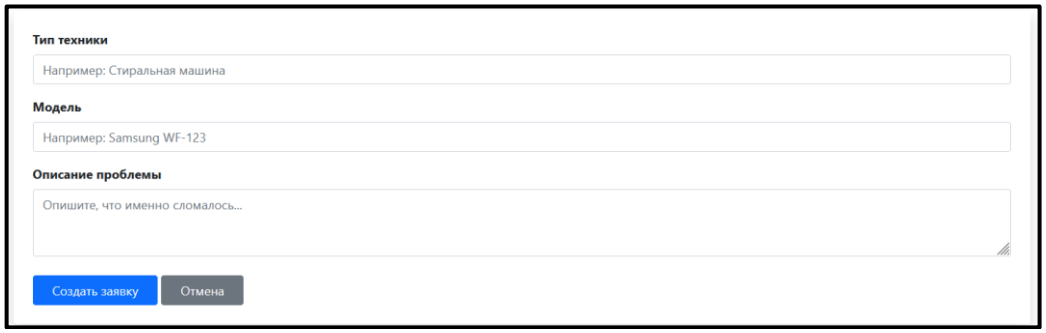

Рисунок 4.8 – Страница добавления новой заявки

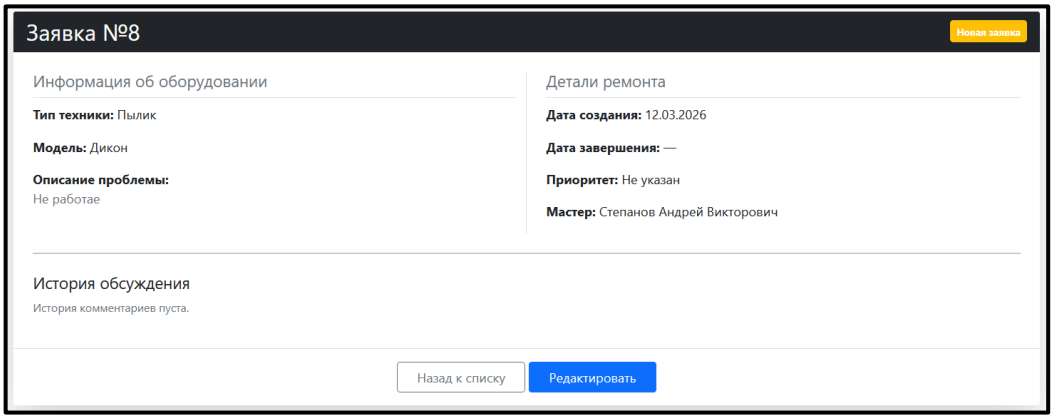

Рисунок 4.9 – Страница с информацией заявки

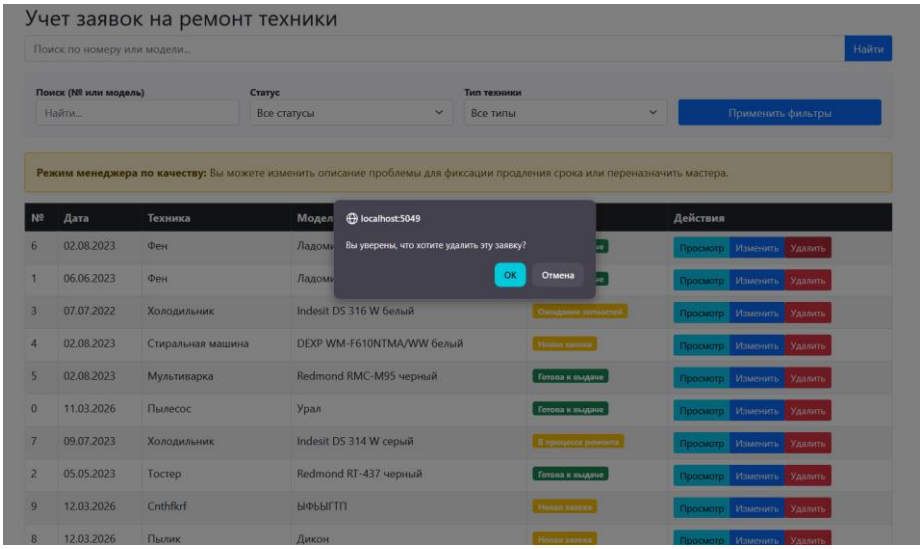

Рисунок 4.10 – Удаление заявки менеджером

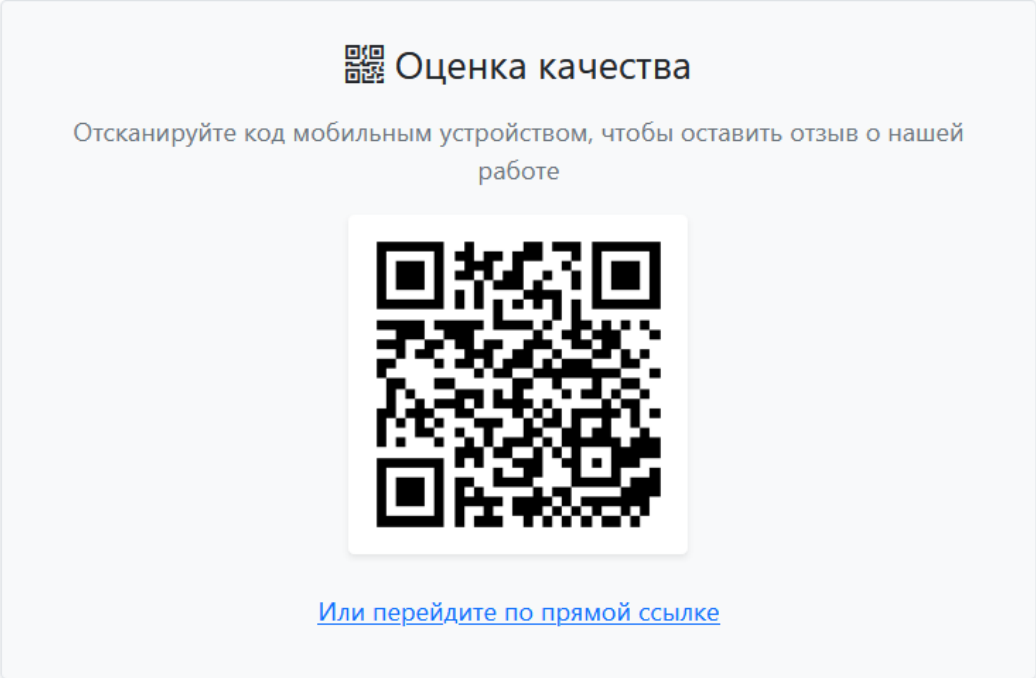

Рисунок 4.11 – Страница отзыва

5 Описание дипломного проекта

5.1 Обоснование выбора инструментальных средств разработки

Для разработки мобильного приложения был выбран следующий технологический стек:

Kotlin – основной язык программирования;

Jetpack Compose – современный декларативный UI-фреймворк для создания пользовательского интерфейса;

MVVM (Model-View-ViewModel) – архитектурный паттерн для разделения бизнес-логики и визуального представления;

Retrofit и OkHttp – библиотеки для осуществления сетевых запросов и взаимодействия с внешним REST API;

Room (SQLite) – система управления локальными базами данных для надежного хранения информации на устройстве;

Jetpack DataStore – инструмент для асинхронного и безопасного хранения пользовательских настроек (сессий, тем оформления, параметров уведомлений);

Android Studio – интегрированная среда разработки (IDE).

Для разработки клиентской части приложения выбран язык Kotlin, который является официальным и наиболее современным стандартом для разработки под операционную систему Android. Выбор Jetpack Compose в качестве UI-фреймворка обусловлен его декларативным подходом, который позволяет описывать интерфейс в виде функций, что значительно ускоряет создание гибких, реактивных и адаптивных экранов.

Для взаимодействия с внешним сервером агрегатора расписаний (Mobifitness API) используется библиотека Retrofit, обеспечивающая надежную обработку HTTP-запросов и автоматическую сериализацию JSON-ответов в объекты приложения. В качестве локальной базы данных применяется библиотека Room — надстройка над SQLite, которая предоставляет удобный объектно-ориентированный интерфейс доступа к данным с проверкой корректности SQL-запросов на этапе компиляции.

Разработка проекта выполняется в среде Android Studio, которая обеспечивает полный цикл создания мобильных приложений, содержит встроенные инструменты профилирования, отладки и эмуляции различных устройств.

5.2 Дизайн приложения

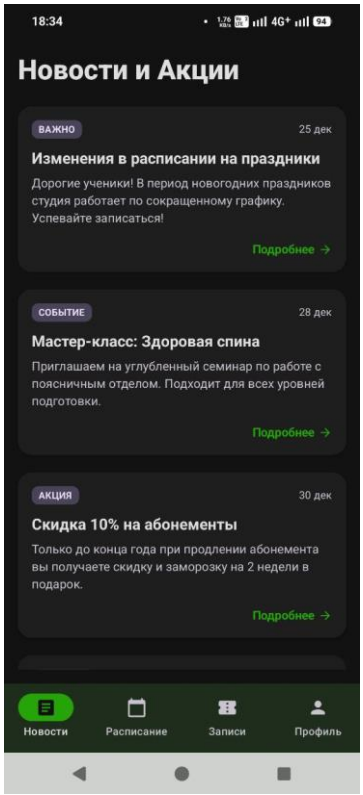

Рисунок 5.1 – Сраница новостей

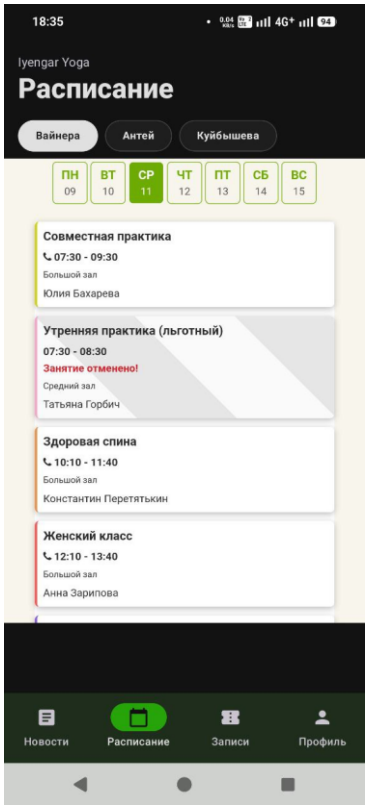

Рисунок 5.2 – Страница вывода расписания через веб-интерфейс

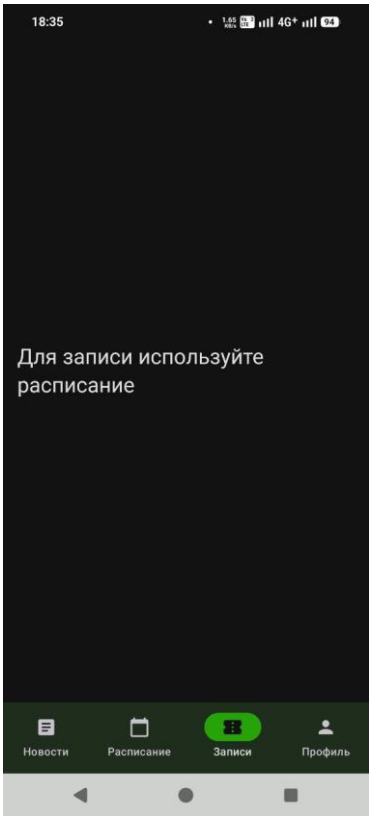

Рисунок 5.3 – Страница отображения записей

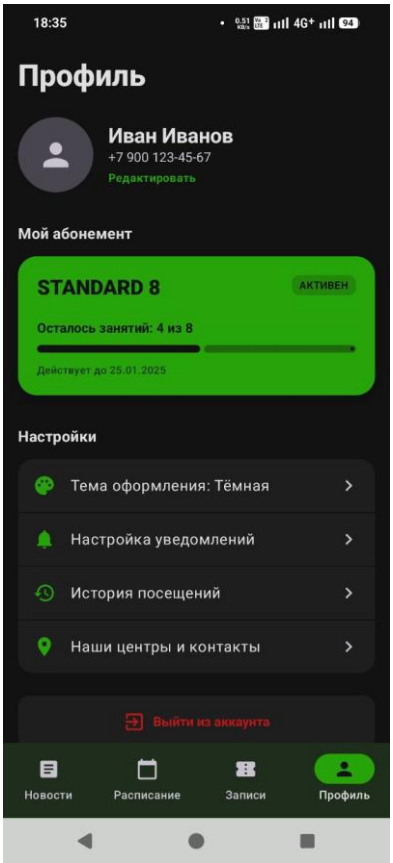

Рисунок 5.4 – Страница профиля

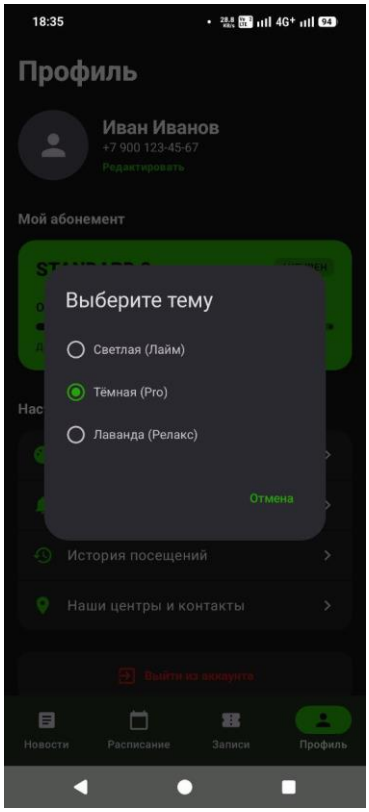

Рисунок 5.5 – Окно выбора цветового оформления

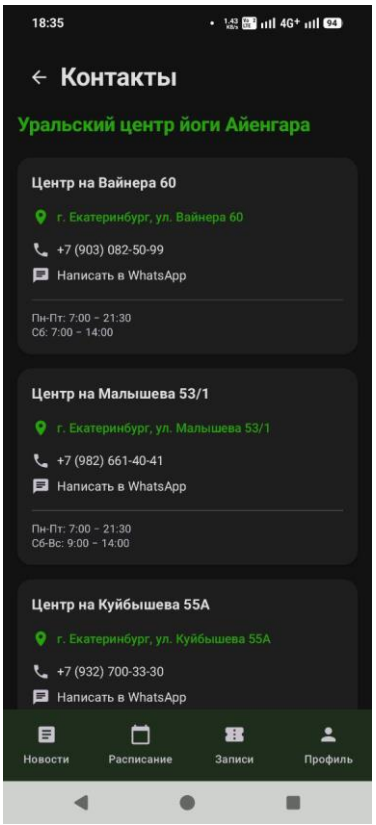

Рисунок 5.6 – Страница контактной информации

Заключение

Разработанный программный модуль для учёта заявок на ремонт бытовой техники представляет собой полнофункциональное решение, автоматизирующее ключевые процессы сервисного центра. Система обеспечивает удобный интерфейс для всех категорий пользователей и предоставляет широкий набор функций для эффективного управления заявками.

В ходе выполнения работы были решены следующие задачи:

- проведен анализ предметной области и определены требования к системе;
- спроектирована структура базы данных в третьей нормальной форме с обеспечением ссылочной целостности;
- разработан графический интерфейс пользователя с учетом разделения прав доступа;
- реализован функционал для администратора;
- реализован функционал для менеджера;
- реализован функционал для клиента;
- выполнена интеграция с СУБД PostgreSQL;
- реализована система уведомлений об изменении статуса заявок;
- добавлена возможность формирования расширенной статистики и экспорта отчетов;
- оптимизирована работа с базой данных и взаимодействие с пользователями.

Таким образом, разработанный программный модуль полностью соответствует поставленным задачам и готов к эксплуатации в сервисном центре по ремонту бытовой техники.

Приложение А

Create.cshtml

```
@model Uset_zayavok.Models.Request

<div class="card shadow-sm mt-3">
    <div class="card-header bg-dark text-white">
        <h4 class="mb-0">Новая заявка на ремонт</h4>
    </div>
    <div class="card-body">
        <form asp-action="Create" method="post">
            <div class="row">
                <div class="col-md-6 mb-3">
                    <label class="form-label">Тип оборудования</label>
                    <input asp-for="Hometechtype" class="form-control"
placeholder="Например: Фен" required />
                </div>
                <div class="col-md-6 mb-3">
                    <label class="form-label">Модель</label>
                    <input asp-for="Hometechmodel" class="form-control"
placeholder="Например: Dyson v10" required />
                </div>
            </div>

            <div class="mb-3">
                <label class="form-label">Описание проблемы</label>
                <textarea asp-for="Problemdescryption" class="form-control"
rows="3" placeholder="Что сломалось?"></textarea>
            </div>

            <div class="mb-3">
                <label class="form-label">Клиент</label>
                <select asp-for="Clientid" class="form-select" required>
                    <option value="">-- Выберите из списка --</option>
                    @foreach (var client in ViewBag.Clients)
                    {
                        <option value="@client.Userid">@client.Fio</option>
                    }
                </select>
            </div>

            <div class="mt-4">
                <button type="submit" class="btn btn-success">Добавить в
базу</button>
                <a asp-action="Index" class="btn btn-outline-secondary">Отмена</a>
            </div>
        </form>
    </div>
      </div>
```

Edit.cshtml

```
@model Uset_zayavok.Models.Request

<div class="container mt-4">
    <div class="card shadow">
        <div class="card-header bg-info text-white">
            <h4>Редактирование заявки №@Model.Requestid</h4>
        </div>
        <div class="card-body">
            <form asp-action="Edit">
                <input type="hidden" asp-for="Requestid" />

                <div class="row">
                    <div class="col-md-6 mb-3">
                        <label class="form-label font-weight-bold">Техника</label>
                        <input class="form-control" value="@Model.Hometechtype
(@Model.Hometechmodel)" readonly />
                    </div>
                    <div class="col-md-6 mb-3">
                        <label class="form-label font-weight-bold">Статус
работы</label>
                        <select asp-for="Requeststatus" class="form-select border-
primary">
                            @foreach (var status in ViewBag.Statuses)
                            {
                                <option value="@status">@status</option>
                            }
                        </select>
                    </div>
                </div>

                <div class="mb-3">
                    <label class="form-label font-weight-bold">Описание
проблемы</label>
                    <textarea asp-for="Problemdescryption" class="form-control"
rows="3"></textarea>
                </div>

                <div class="mb-3">
                    <label class="form-label font-weight-bold">Запчасти / Заметки
мастера</label>
                    <input asp-for="Repairparts" class="form-control"
placeholder="Какие детали заменены..." />
                </div>

                <div class="mt-4 border-top pt-3">
                    <button type="submit" class="btn btn-success px-
4">Сохранить</button>
                    <a asp-action="Index" class="btn btn-secondary px-
4">Отмена</a>
                </div>
            </form>
        </div>
    </div>
      </div>
```

Employees.cshtml

```
@model IEnumerable<Uset_zayavok.Models.User>

<div class="container mt-4">
    <div class="d-flex justify-content-between align-items-center mb-3">
        <h2>Список сотрудников</h2>
    </div>

    <table class="table table-striped table-hover shadow-sm">
        <thead class="table-dark">
            <tr>
                <th>ФИО</th>
                <th>Должность</th>
                <th>Номер телефона</th>
                <th>Логин</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var user in Model)
            {
                <tr>
                    <td>@user.Fio</td>
                    <td>
                        <span class="badge @(user.Type == "Администратор" ? "bg-
danger" : "bg-info text-dark")">
                            @user.Type
                        </span>
                    </td>
                    <td>@user.Phone</td>
                    <td>@user.Login</td>
                </tr>
            }
        </tbody>
    </table>
      </div>
```

Index.cshtml

```
@model IEnumerable<Uset_zayavok.Models.Request>

@{
    ViewBag.Title = "Список заявок";
}

<div class="container">
    <h2>Учет заявок на ремонт техники</h2>

    <form method="get" class="mb-3">
        <div class="input-group">
            <input type="text" name="search" class="form-control"
placeholder="Поиск по номеру или модели..." />
            <button type="submit" class="btn btn-primary">Найти</button>
        </div>
    </form>

    <table class="table table-striped table-bordered">
        <thead class="table-dark">
            <tr>
                <th>№</th>
                <th>Дата</th>
                <th>Техника</th>
                <th>Модель</th>
                <th>Статус</th>
                <th>Действия</th>
```

```
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model)
            {
                <tr>
                    <td>@item.Requestid</td>
                    <td>@(item.Startdate?.ToShortDateString())</td>
                    <td>@item.Hometechtype</td>
                    <td>@item.Hometechmodel</td>
                    <td>
                        <span class="badge @(item.Requeststatus == "Готова к
выдаче" ? "bg-success" : "bg-warning")">
                            @item.Requeststatus
                        </span>
                    </td>
                    <td>
                        @if (User.IsInRole("Мастер") ||
User.IsInRole("Администратор"))
                        {
                            <a asp-action="Edit" asp-route-id="@item.Requestid"
class="btn btn-sm btn-info">Редактировать</a>
                        }
                        else
                        {
                            <span class="text-muted">Только просмотр</span>
                        }
                    </td>
                </tr>
            }
        </tbody>
    </table>
      </div>
```

Status.cshtml

```
@model Dictionary<string, int>

<div class="container mt-4">
    <h2 class="mb-4 text-center">Статистика и статусы заявок</h2>

    <div class="alert alert-info shadow-sm text-center">
        <h4>Всего зарегистрировано заявок:
<strong>@ViewBag.TotalRequests</strong></h4>
    </div>

    <div class="row mt-4">
        @foreach (var status in Model)
        {
            <div class="col-md-4 mb-3">
                <div class="card border-0 shadow-sm h-100">
                    <div class="card-body text-center">
                        <h5 class="card-title text-muted">@status.Key</h5>
                        <p class="display-4 font-weight-bold text-
primary">@status.Value</p>

                        @{

                            string progressColor = status.Key == "Готова к
выдаче"? "bg-success" :
                                                status.Key == "В процессе
ремонта" ? "bg-warning" : "bg-secondary";
                        }
                        <div class="progress" style="height: 10px;">
```

```
                                <div class="progress-bar @progressColor"
role="progressbar"
                                     style="width: @(ViewBag.TotalRequests > 0 ?
(status.Value * 100 / ViewBag.TotalRequests) : 0)%">
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            }
        </div>
        <div class="row text-center mb-4">
            <div class="col-md-6">
                <div class="card bg-success text-white">
                    <div class="card-body">
                        <h3>@ViewBag.CompletedCount</h3>
                        <p>Выполнено заявок </p>
                    </div>
                </div>
            </div>
            <div class="col-md-6">
                <div class="card bg-warning text-dark">
                    <div class="card-body">
                        <h3>@ViewBag.AverageTime дней</h3>
                        <p>Среднее время ремонта </p>
                    </div>
                </div>
            </div>
        </div>

        <div class="text-center mt-5">
            <a asp-action="Index" class="btn btn-outline-dark px-5">Перейти к полному
списку</a>
        </div>
</div>
```

ListController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;
using Uset_zayavok.Models;


namespace Uset_zayavok.Controllers
{
    [Authorize]
    public class ListController : Controller
    {
        private readonly PostgresContext _context;

        public ListController(PostgresContext context)
        {
            _context = context;
        }

        public IActionResult Index(string search)
        {

            var userIdClaim = User.FindFirst("UserId")?.Value;
            var userRole = User.FindFirst(ClaimTypes.Role)?.Value;
```

```
            if (string.IsNullOrEmpty(userIdClaim)) return
RedirectToAction("Login", "Account");

            int currentUserId = int.Parse(userIdClaim);


            var requests = _context.Requests.Include(r => r.Client).AsQueryable();

            if (userRole == "Заказчик")
            {

                requests = requests.Where(r => r.Clientid == currentUserId);
            }



            if (!string.IsNullOrEmpty(search))
            {
                requests = requests.Where(r => r.Hometechmodel.Contains(search) ||
                                               r.Requestid.ToString() == search);
            }

            return View(requests.ToList());
        }

        public IActionResult Create()
        {
            ViewBag.Clients = _context.Users.Where(u => u.Type ==
"Заказчик").ToList();
            return View();

        }
        [HttpPost]
        public IActionResult Create(Request request)
        {
            int nextId = _context.Requests.Any() ? _context.Requests.Max(r =>
r.Requestid) + 1 : 1;
            request.Requestid = nextId;

            request.Startdate = DateOnly.FromDateTime(DateTime.Now);
            request.Requeststatus = "Новая заявка";

            _context.Requests.Add(request);
            _context.SaveChanges();

            return RedirectToAction("Index");
        }

        [HttpGet]
        public IActionResult Status()
        {

            var stats = _context.Requests
                .GroupBy(r => r.Requeststatus)
                .Select(g => new { Status = g.Key ?? "Не указан", Count =
g.Count() })
                .ToDictionary(x => x.Status, x => x.Count);

            ViewBag.CompletedCount = _context.Requests
        .Count(r => r.Requeststatus == "Готова к выдаче" || r.Requeststatus ==
"Завершена");


            ViewBag.TotalRequests = _context.Requests.Count();
```

```
            ViewBag.TypeStats = _context.Requests
                .GroupBy(r => r.Hometechtype)
                .Select(g => new { Type = g.Key ?? "Не определено", Count =
g.Count() })
                .ToDictionary(x => x.Type, x => x.Count);


            var completed = _context.Requests
                .Where(r => r.Startdate != null && r.Completiondate != null)
                .ToList();

            double avgDays = 0;
            if (completed.Any())
            {
                avgDays = completed.Average(r =>
                    (r.Completiondate.Value.ToDateTime(TimeOnly.MinValue) -
                     r.Startdate.Value.ToDateTime(TimeOnly.MinValue)).TotalDays);
            }
            ViewBag.AverageTime = Math.Round(avgDays, 1);

            return View(stats);
        }

        [Authorize(Roles = "Мастер,Администратор")]
        public IActionResult Edit(int id)
        {

            var request = _context.Requests.Find(id);
            if (request == null) return NotFound();
            ViewBag.Statuses = new List<string>
             {
            "Новая заявка",
            "В процессе ремонта",
            "Готова к выдаче",
            "Завершена"
            };

            return View(request);
        }
        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Edit(int id, Request request)
        {
            if (id != request.Requestid) return NotFound();

            try
            {
                var dbEntry = _context.Requests.Find(id);
                if (dbEntry == null) return NotFound();
                dbEntry.Requeststatus = request.Requeststatus;
                dbEntry.Problemdescryption = request.Problemdescryption;
                dbEntry.Repairparts = request.Repairparts;
                if (request.Requeststatus == "Готова к выдаче")
                {
                    dbEntry.Completiondate = DateOnly.FromDateTime(DateTime.Now);
                }

                _context.SaveChanges();
                return RedirectToAction(nameof(Index));
            }
            catch
            {
                return View(request);
```

```
            }
        }
        [HttpGet]
        public IActionResult Employees()
        {
            var staff = _context.Users
                .Where(u => u.Type != "Заказчик")
                .ToList();

            return View(staff);
        }
    }

      }
```

Login.cshtml

```
@{
    Layout = "_Layout";
    ViewData["Title"] = "Вход в систему";
}

<div class="row justify-content-center mt-5">
    <div class="col-md-4">
        <div class="card shadow">
            <div class="card-header bg-primary text-white text-center">
                <h4>Авторизация</h4>
            </div>
            <div class="card-body">
                <form asp-action="Login" method="post">
                    @if (ViewBag.Error != null)
                    {
                        <div class="alert alert-danger">@ViewBag.Error</div>
                    }
                    <div class="mb-3">
                        <label class="form-label">Логин</label>
                        <input type="text" name="login" class="form-control"
required />
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Пароль</label>
                        <input type="password" name="password" class="form-
control" required />
                    </div>
                    <button type="submit" class="btn btn-primary w-
100">Войти</button>
                </form>
            </div>
        </div>
    </div>
      </div>
```